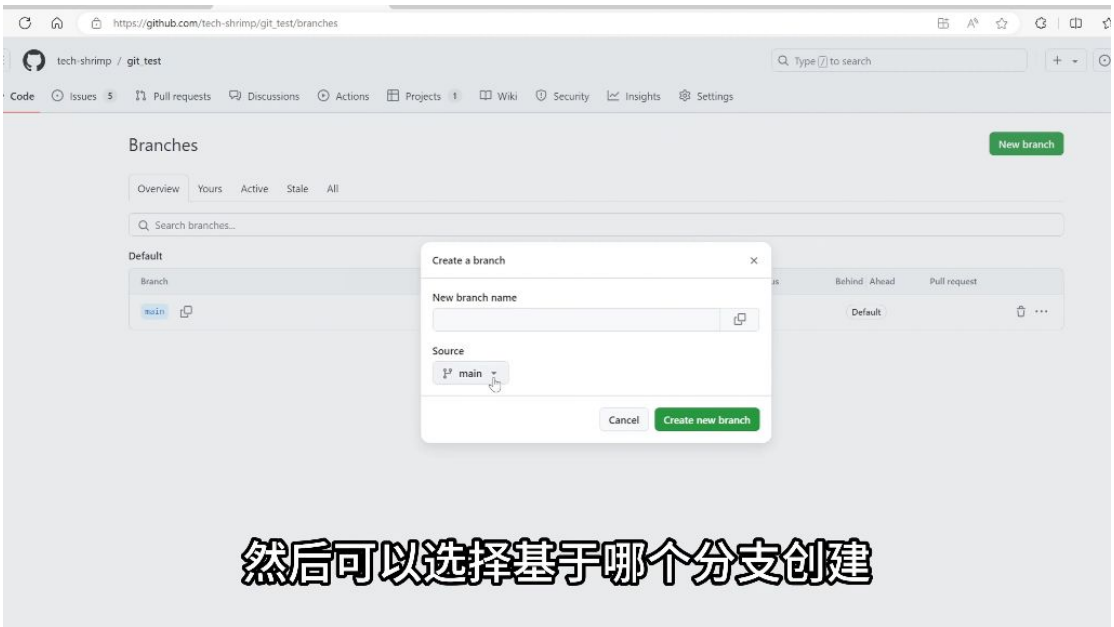
-- 以下为AI生成的图文笔记内容 --

一、*GitHub Desktop与分支操作00:00*

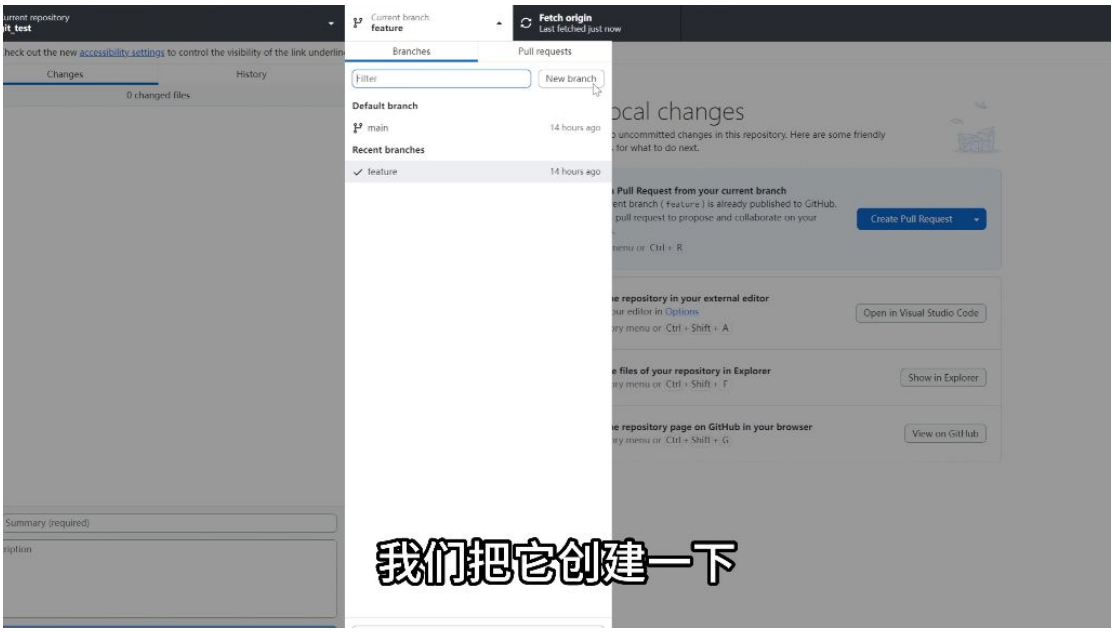
1. 创建分支 00:13

1）网页创建分支 00:28

- 

- **操作步骤**：
  - 进入项目首页，点击分支按钮查看所有分支
  - 点击右上角"New branch"创建新分支
  - 选择基于哪个分支创建（如main分支）
  - 输入分支名称（如feature）并点击"Create new branch"
- **同步到本地**：
  - 在GitHub Desktop中点击"Fetch origin"获取远端更新
  - 切换分支时点击分支按钮选择目标分支

2）Desktop创建分支 00:52

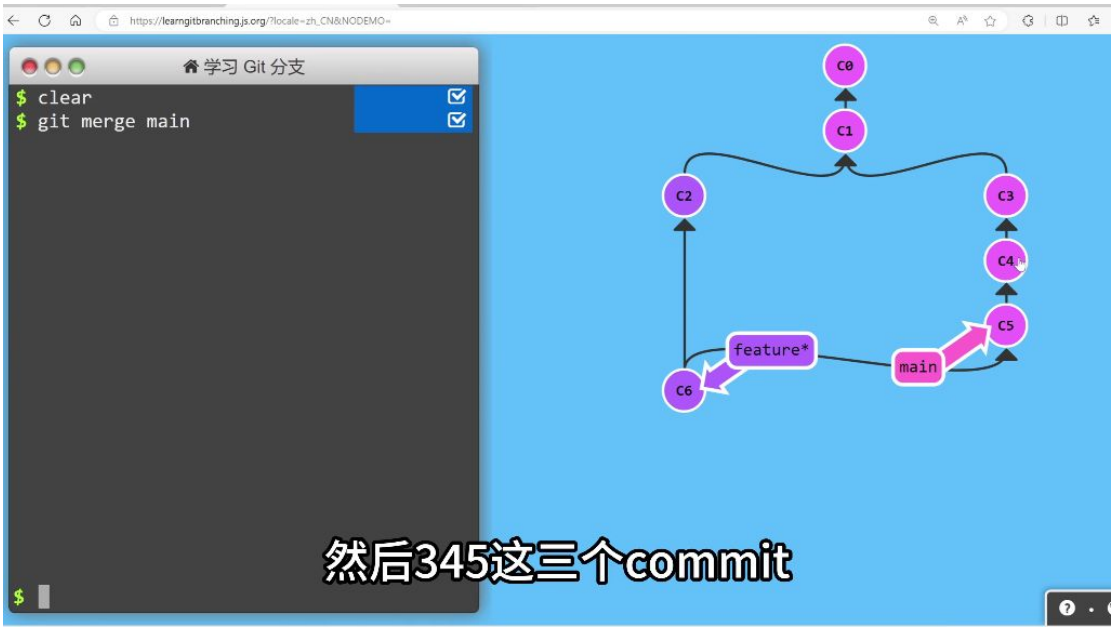
- 

- **操作步骤**：
  - 点击分支按钮选择"Create branch"
  - 选择基于哪个分支创建（如main分支）
  - 输入分支名称（如feature2）
  - 点击"Create branch"
- **推送远端**：
  - 新创建的分支默认只在本地存在
  - 需要点击"Publish branch"推送到远端
  - 验证：在GitHub网页分支界面刷新可看到新分支

2. 分支合并 02:55

1）使用GitHub Desktop合并分支 03:24

- 合并、Squash合并、变基 04:35

- 

  - **普通合并(merge)**：
    - **特点**：创建新的merge commit，保留所有原始提交记录
    - **优点**：保证分支可溯源，适合多人协作
    - **缺点**：多一次merge记录，历史可能显得杂乱
    - **命令**：git merge main
  - **Squash合并**：
    - **特点**：将多个提交压缩成一个新提交
    - **优点**：历史记录更清爽干净
    - **缺点**：原始提交记录被合并，不便追溯
    - **命令**：git merge --squash main
  - **变基(rebase)**：
    - **特点**：将当前分支变基到目标分支，形成线性历史
    - **优点**：减少merge记录，历史更整洁
    - **缺点**：必须使用强制推送，可能覆盖他人代码
    - **命令**：git rebase main

- **注意**：仅适用于私有分支，多人协作分支禁用

- 三种操作的不同 09:25

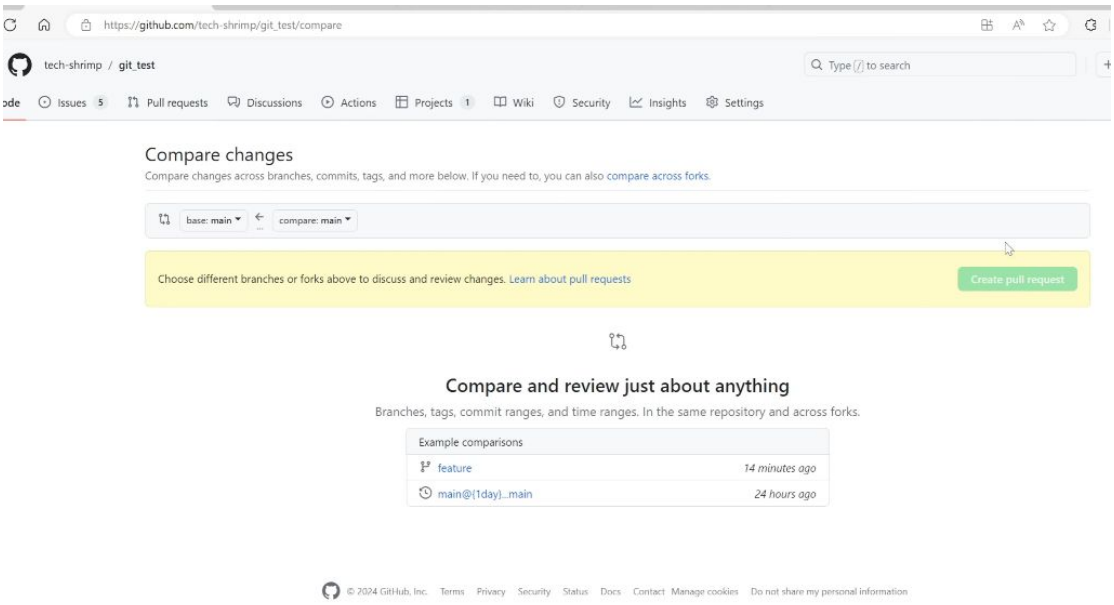
14. 分支合并 rebase，merge，squash merge /

操作	rebase	merge	squash merge
特点	只有线性提交记录	会出现所有提交记录，包括merge记录	只出现一条merge记录
优点	减少一次merge记录	保证分支可溯源	历史记录更加清爽干净
缺点	必须使用强推	多一次merge记录	历史记录被合并了到了一起
应用场景	在私有分支上可以用 rebase	多人协作分支必须merge，向主干分支合并必须merge	多人协作分支，需要保持记录清爽

用来总结这三种操作的不同

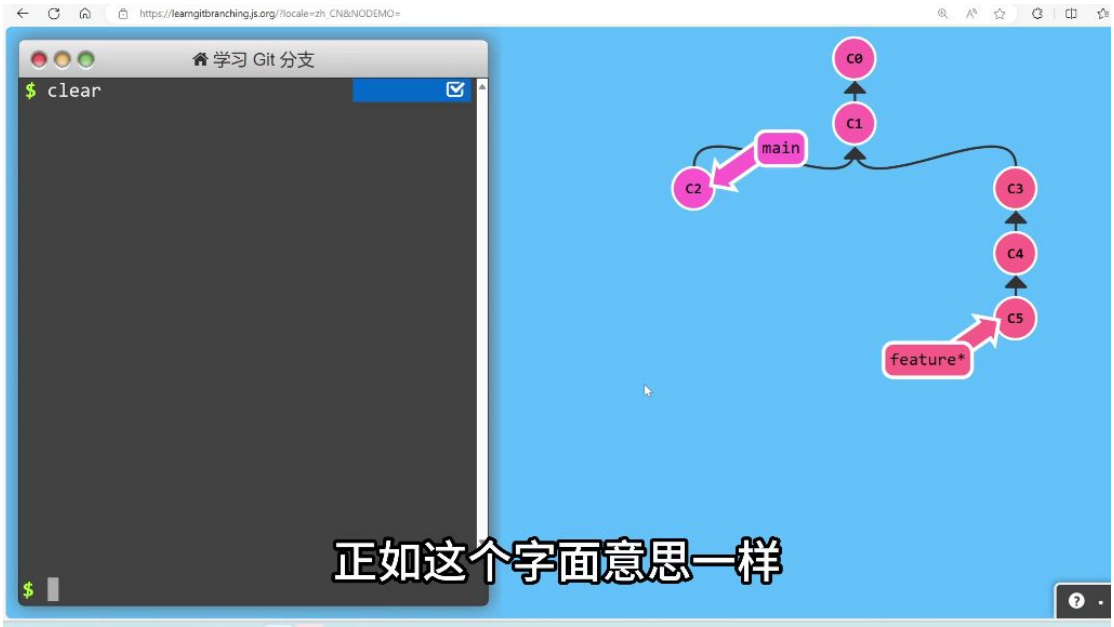
- **应用场景**：
  - Rebase：私有分支保持历史整洁
  - Merge：向主干分支合并或多人协作分支
  - Squash：多人协作分支需要简化历史记录
- **强制推送**：
  - 变基后必须使用git push -f强制推送
  - 强制推送会改变远程历史，可能影响他人工作

2）网页端合并 10:50

- 

- **推荐方式**：
  - 网页端只推荐使用普通合并和Squash合并
  - 避免在网页端使用变基操作
- **可视化工具**：
  - 通过Insights → Network查看分支合并状态
  - 图形化展示分支历史和合并关系

3. Cherry Pick 11:33

- 

- **概念**：选择性合并特定提交到目标分支

- **命令行操作**：
  - 切换到目标分支：git switch main
  - 执行cherry-pick：git cherry-pick c3 c4
- **Desktop操作**：
  - 在源分支选中要合并的提交（可多选）
  - 右键选择"Cherry pick commits to..."
  - 选择目标分支完成合并
- **注意事项**：
  - Desktop操作需要先切换到输出commit的分支
  - 命令行操作需要先切换到接受合并的分支
  - 合并后需要推送更改到远端

二、知识小结

知识点	核心内容	关键区别/特点	应用场景	注意事项
分支创建	两种创建方式： 1. GitHub网站创建后同步 2. Desktop本地创建后推送	- 网站创建自动同步 - 本地创建需手动发布	功能开发/版本维护	创建后需fetch获取远端更新
分支删除	两种删除方式： 1. 网页端删除(仅远端) 2. Desktop删除(可选本地+远端)	- 网页操作仅影响远端 - Desktop可批量删除	分支生命周期管理	**注意**本地与远端同步状态
普通合并(Merge)	git merge命令 - 保留所有原始提交 - 生成新merge commit	**优点**：完整可追溯 **缺点**：多一条合并记录	多人协作分支 主干分支合并	必须使用场景：主干分支合并

压缩合并 (Squash Merge)	git merge --squash - 压缩多个提交为单个 - 生成新 squash commit	**优点**：历史清爽 **缺点**：丢失细节记录	多人协作分支 需简化历史时	不适用于需要完整审计的场景
变基合并 (Rebase)	git rebase命令 - 重写提交历史 - 不生成额外commit	**优点**：线性历史 **缺点**：必须强制推送	**私有分支**维护 个人功能分支	**严禁**在多人协作分支使用
优选提交 (Cherry Pick)	选择性合并特定提交 - git cherry-pick <commit> - Desktop需先切到源分支	精准移植代码变更 不合并完整分支	热修复补丁 部分功能移植	需注意提交依赖关系
合并策略对比	三合一对比维度： 1. 历史记录形式 2. 推送要求 3. 协作安全性	**强制推送**是rebase最大风险点 **squash**平衡简洁与安全	根据分支性质选择策略	主干分支**禁止**rebase